Documentation for the use of Palabos

Gabriel Rochette

January, 2026

Abstract

This is a documentation for the use of the modified version of Palabos I have developed during my 2025/2026 internship. It contains practical information as to:

1. The architecture of the folder.
2. The different modification of interest I have made.
3. How to run the code.
4. How to deploy it on cluster.
5. Different notions concerning unit conversion.

1 Introduction

This software is a slightly modified version of Palabos. The key modifications include:

1. Addind pressure computation at the Lattice level.
2. Adding HDF5 output writing for 2D problems.
3. Changing the way boolean masks are read.

The other modifications applied do not concern the code itself but rather its use:

1. Adding the reading of input files to allow the user to configure most of the problem easily without having to dive into the C++ code.
2. Arranging the folder organisation to facilitate reading and access.
3. Writing additional Python files to automize the process of geometry generation for AoA sweep.
4. Preparing batch submission script for the Delta 2 cluster.

2 The Architecture of the folder

Everything happens in the 'airfoil' subfolder. There, you can find three main folders. The **include** folder contains header files that are no longer in use and can be overlooked. It was kept to ensure proper functioning of Palabos in early development phases. The **geometry** folder contains:

1. .dat files containing NACA shapes, obtained directly from the NACA website. It was copy/pasted from the corresponding area of the website, and **not downloaded**. Indeed, the python code generating the geometry will create a polynomial shape by connecting each point in *the order they appear*, meaning it's particularly important to ensure the .dat file stores point in the correct order. To ensure the polynomial shape is correctly closed, the first point of the shape is manually added at the very end of the list (which effectively closes the loop).
2. The 'geometryGenerator.py' script. This is where the user can generate the geometry of interest by:

(a) Specifying the dimension of the domain: *scale_x* and *scale_y* correspond to the number of chord length along the x and y direction (by default the shape defined in the .dat file will be scaled to occupy one chord length along the x direction).

(b) Specifying the lattice resolution: *lattice_resolution* corresponds to the number of lattices per chord length (by default 150).

(c) Specifying the .Dat file of interest (to define the geometry through the polynomial shape).

(d) Specifying the AoA range of interest.

(e) Specifying the porosity parameters (number of pores, spacing, relative angle...).

Note that at this point an angle sweep on the porosity relative angle is not yet implemented. Also, for a domain of size $N \times M$, Palabos require inputing a domain of size $(N+1) \times (M+1)$, hence the padding at the end of the code. This effect is accounted for in the main simulation code.

3. The 'palabosGeometryViewer.py', which is aimed at checking the .dat files generated work properly by reproducing the exact steps palabos uses to read a .dat file and plotting them for human visualisation. You can use this to ensure you have properly created the geometry files. The user has to manually input the same data as previosuly when defining the domain size and resolution (*scale_x*, *scale_y*, and *lattice_resolution*).

4. The 'generateXml.py' that generates all correct xml parameters file from a corresponding AoA list. The user only has to move those to the correct path in the main folder, as discussed in the next part.

Most of the fun happens in the **main** folder. The architecture is as such:

```
main/
├── build/
├── geometry/
│   └── geometry_AoA.dat
├── Results/
│   └── AoA/
├── test/
├── tmp/
├── xmlFiles
│   └── parameters_AoA.xml
├── airfoil (executable)
├── airfoil.cpp
└── other files
```

Where all references to 'AoA' have to be understood as loops on AoA values specified in the earlier phase. Lets's break it down.

- The build/ folder contains the files used for building the executable. We will dive into more detail in further section, but basically, this is were the cmake and make command will take place, **resulting in the creation of the 'arifoil' executable from the 'airfoil.cpp' code**.

- The geometry/ folder contains all the .dat file, identified per AoA, that will be called in the batch script. Their paths are stored in the parameters_AoA.xml file of corresponding AoA, and do not have to be manipulated at any point other after having been generated by the previously mentioned python code and dropped manually in this folder.

- The Results/ folder contains subfolders identified by angle of attack. The xml parameter file also contains information to locate these folders, and the code will automatically save simulation results there.

- The test/ and temp/ folders are simply there for development purposes in case you want to modify the code. You can use the test/ folder to store test data, and the temp/ folder to store test results.

- The xmlFiles/ folder is where the parameter files are stored. The C++ code takes an xml file as input, and does not need any additional information. You can find example architecture in appendix A. It contains: the relative path from the executable to the geometry of interest, the lattice resolution (named 'N'), and the scale x and scale y parameters (named 'lx' and 'ly'). Passing this as input, the code will automatically adapt the domain size of interest, find the correct result folder thanks to the angle of attack, and read the correct geometry from the specified location. The generation of these xml files has been automated, as discussed previously.
- Additionally , some other files are present, such as bash scripts automating the cmake and make commands to gain a bit of time when toying with the main C++ code and recompiling.

3 Modification of interest

As explained, I have spent a bit of time modifying both the source code and the main executable to produce an easy to use 2D solver for CFD problems. I will not dive into the details of the source code I have modified, but simply explain what new features have been added for you to use.

- When using a 2D lattice, it is now possible to call the 'computePressure' function to retrieve the pressure on the overall lattice for post-processing purposes.
- It is now possible to easily write HDF5 output files for 2D lattices by using the canonical syntax of palabos, simply using the 'ParallelXmdfDataWriter2D' class instead of the 3D equivalent.

Additionally to these two new features, I have also modified the way Palabos uses boolean masks. Initially, Palabos' documentation points out to a specific way of defining geometry from boolean masks. However, this specified way only supports the most straightforward dynamics (classical bounce backs etc.) and does not work with momentum exchange. I have written a new way of importing a boolean mask as a MaskShapeDomain2D (the most generic class in Palabos that works with any dynamic), by defining a constructor from the boolean mask directly. The comparison of Palabos' classical method with mine is presented in appendix B.

Other modifications such as adding parameter reading, using MRT etc. are somewhat painstaking to implement but straightforward. Once implemented they do not have to be modified, so explaining the procedure is out of the scope of this documentation.

The overall modifications or additions I have made have been mentioned in the folder architecture section, and will be detailed in the next one.

4 Running the code

Here is a detailed procedure to run the code on your machine. I am using WSL, version: Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64).

1. Go to your folder of interest. Run 'git clone https://github.com/GabrielRCS/Cranfield-Porous-Airfoil.git'
2. Move to './airfoil/geometry/'
3. There, open python files of interest and fill in the values you are interested in.
4. Run the python files for the geometry and xml files, and move them to the geometry and xmlFiles folders in the './airfoil/main/' folder. You can erase any existing files if they do not have any interest for you.
5. Go to the './airfoil/main/'
6. Open the 'airfoil.cpp' executable. Modify the number of iterations. Choose for which iterations ('one every X iterations') you want to save the different informations (gif, vti files, and lift and drag values). (This will be made from the xml file in later versions)
7. Go to the Results/ folder, and create the appropriate result folder based on the angle of attack you used in your xml file. Go back to the main/ folder.

8. Remove the airfoil executable, **not the airfoil.cpp file**! You will have to recompile it with your versions of the different modules.

9. Move to the './build' folder. Remove any file present there, and run 'cmake ..'. Download additional modules if prompted, such as OpenMPI, HDF5 writing modules, or any other module that could be hindering the compilation. Once it is complete, run the 'make' command ('make -j' if you want to use parallel compilation for significant gain in time). This will recreate the executable with the correct module versions.

10. Move back to the main folder. Now, you can simply run 'mpirun -np {#of mpi process} airfoil {your parameters.xml File}

11. The code will write results in the AoA folder corresponding to your input. You will find gifs written at different iterations (that you can easily convert to a longer animation using linux's 'convert' function), one vti file containing pressure, density, vorticity and different informations on velocity, saved at the iterations you chose in the previous steps. You will also find two text files containing the Drag and Lift coefficients saved at the iterations you chose.

5 Example use case

Follow these steps to run an example.

1. git clone https://github.com/GabrielRCS/Cranfield-Porous-Airfoil.git
2. cd ./Cranfield-Porous-Airfoil/airfoil/main/
3. rm airfoil
4. rm -rf build/
5. mkdir build/
6. cd build/
7. cmake ..
8. make -j
9. cd ..
10. mpirun -np 12 airfoil ./xmlFiles/parameters_0.xml

The code will now execute. You can visualize results by going into ./Results/0/ and looking into the various files of interest.

6 Deploying the code on Cranfield's Delta 2 cluster

Deploying the code works exactly as previously, with one exception: you have to manually load the modules in your environement to properly compile the airfoil executable so that the batch submission can work on the machines.

Here are the list of modules you'll have to load in your environment before recompiling (cmake and make steps):

1. gompi/2021a
2. DHF5/1.10.7-gompi-2021a
3. ImageMagick/7.10.0-4-GCCore-11.2.0
4. GCCcore/12.2.0

Upon loading these modules, recompile the executable properly on the cluster's environment, and the batch submission will work. You can find a script example of a batch job submission in the ./Cranfield-Porous-Airfoil/airfoil/ folder.

7 Some mathematical notions for postprocessing

One has to bear in mind that LBM manipulates only values in Lattice Units. Therefore, any sucessful post-processing of those values will have to implement proper unit conversions. Here is the list of conversions to implement, with the "real" subscript denoting values expressed in SI units, while "lattice" refers to values extracted from the code, in "lattice" units.

- $\rho_{real} = \rho_{lattice}$ (the very nature of the LBM method preserves correct values for density. The subscript is dropped in further equations)
- $t_{real} = t_{lattice} * dt$ (lattice time is the number of iteration)
- $u_{real} = u_{lattice} \times \frac{dx}{dt}$ (dx is the distance between two lattices, and with the current code, one has $dx = dy$)
- When running the getInternalStatistics.getSum from Palabos, we obtain a value corresponding to the sum of all the momentum exchanged by the fluid during the previous iteration of time dt. It thus corresponds to the variation of its momentum during this iteration. We have: $internal\_statistics = \Delta p_{lattice} = \rho \Delta u_{lattice}$ (indeed, this value sums all momentum variation calculated on each cell from the variation of the particle speed, times local density). Thus, we have: $internal\_statistics = \frac{m}{dx^2.dz} \times \frac{dt}{dx} \times \Delta u_{real}$ (by keeping in mind that $dx = dy$), and thus, by using $F_{real} = m \times \frac{\Delta u_{real}}{dt}$:

$$internal\_statistics = F_{real} \times \frac{dt^2}{dx^3.dz}$$

- We can simply the previous formula. As we are working in 2D, we take $dz = 1$. This yields: $F_{real} = internal\_statistics \times \frac{dx^3}{dt^2}$
- Then, it follows from the definition of Lift / Drag coefficients that:

$$C = 2\frac{F_{real}}{\rho S u_{real,ref}^2} = 2\frac{internal\_statistics}{\rho S u_{real,ref}^2} \times \frac{dx^3}{dt^2}$$

  This yields the correct correction factor to apply to the internal statistics sum information (after projection in y/x for Lift/Drag coefficients), $\frac{dx^3}{dt^2}$, to correct the value obtained. One still has tu use correct reference values for the speed and surface (in real units). Note that this correction is already applied in the code, so you needn't modify it (unless you change reference speed or area), but you might be interested in where this comes from in case you want to add aditional post-processing and require similar reasoning.

A Typical architecture for XML files

Example for the file parameters_0.xml:

```
<geometry>
    <location>./geometry/geometry_0.dat</location>
    <AoA>0</AoA>
    <N>150</N>
    <lx>16</lx>
    <ly>8</ly>
</geometry>
```

B Different ways to import a geometry from a boolean mask

B.1 Inherent Palabos Method

The native method from the Palabos documentation (or example codes) loads a boolean mask as follows:

```
MultiScalarField2D<T> boolMask(nx, ny);
plb_ifstream ifile("geometry.dat");
ifile >> boolMask;

defineDynamics(lattice, boolMask, new BounceBack<T,DESCRIPTOR>, true);
```

This method works perfectly fine for classical bounce back mechanics. However, for our problem, we need the use of the momentum exchange bounce back. Since Palabos does not allow the use of this dynamic on objects of type 'MultiScalarField2D', it was important to figure a way to define a geometry from a boolean mask with the generic type for palabos fields: 'MaskShapeDomain2D'.

B.2 Custom class definition

Some examples show definitions of 'MaskShapeDomain2D' from boolean functions (example: a cylinder from the condition $x^2 + y^2 < r^2$). Therefore, it was decided to mimick this behavior by turning the boolean mask into a boolean function, returning 'True' if the (x,y) point corresponds to a 1 in the boolean mask, 0 otherwise. This led to the use of the custom class defined as such:

```
// Class to define a domain from a native 2D boolean mask
template <typename T>
class MaskShapeDomain2D : public DomainFunctional2D {
public:
    // Constructor: takes a native 2D mask (vector of vector of bool)
    MaskShapeDomain2D(const std::vector<std::vector<bool>>& mask_)
        : mask(mask_)
    { }

    // operator(): returns true if the mask is active at a given lattice index
    virtual bool operator()(plint iX, plint iY) const override
    {
        if (iY >= 0 && iY < static_cast<plint>(mask.size()) &&
            iX >= 0 && iX < static_cast<plint>(mask[iY].size()))
        {
            // This is the boolean function
            return mask[iY][iX];
        }
        return false; // out of bounds thus inactive
    }

    // clone() required by Palabos
```

```
    virtual MaskShapeDomain2D<T>* clone() const override
    {
        return new MaskShapeDomain2D<T>(*this);
    }

private:
    std::vector<std::vector<bool>> mask;
};
```

One important thing to note is that the boolean mask is imported as 'std::vector<std::vector<bool>>', and not as a standard Palabos type (like 'MultiScalarField2D<T>'). Indeed, it was particularly important to ensure the code could still be parallelized. Palabos uses a distributed memory architecture (with MPI passing), so initial data has to be split accross different processes. Since Palabos' native types cannot be easily split, the use of native C++ types such as vectors of vectors was the only way to preserve the code's ability to separate data and remain parallelizable.

We can now define the geometry properly:

```
std::vector<std::vector<bool>> boolMask(ny, std::vector<bool>(nx));
for (plint y = 0; y < ny; ++y)
    for (plint x = 0; x < nx; ++x)
        boolMask[y][x] = boolMaskPLB.get(x, y);

defineDynamics(
    lattice, lattice.getBoundingBox(), new MaskShapeDomain2D<T>(boolMask),
    new MomentumExchangeBounceBack<T, DESCRIPTOR>(forceIds));
initializeMomentumExchange(
    lattice, lattice.getBoundingBox(), new MaskShapeDomain2D<T>(boolMask));
```

C Modifying the different parameters linked to iterations

To modify the total number of iterations, or the iterations at which you wish to save the different fields of interest, go to this section:

```
#ifndef PLB_REGRESSION
const T imSave = (T)0.5;
const T vtkSave = (T)20.; // Save VTK files every 20 time units
const T maxT = (T)400.1; // Total simulation time: 4e4 lattice time units, to match Illio
```

And modify:

- maxT for the total number of iterations (note that the value set is the number of seconds, it will be divided by dt to determine the number of iterations).
- imSave for the number of seconds (simulation time) between two gif saves.
- vtkSave for the number of seconds (simulation time) between two vtk saves.